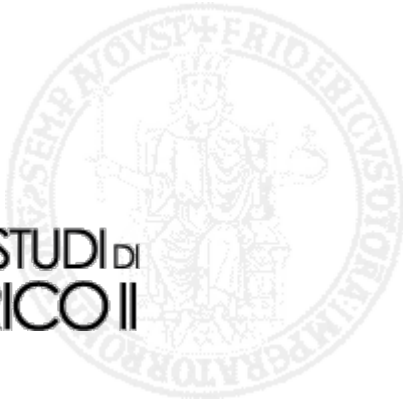


UNIVERSITA' DEGLI STUDI DI
NAPOLI FEDERICO II



Final project work documentation

Classificazione di disturbi neuropsichiatrici tramite algoritmi di Machine Learning

Studente:

Federica Cirillo

Matricola:

M54001498

Indice

1	Introduzione	1
2	Analisi e preprocessing del dataset clinico	3
2.1	Strumenti software utilizzati	3
2.2	Descrizione del dataset	3
2.3	Gestione dei valori mancanti	6
2.4	Individuazione di possibili outliers e eliminazione di colonne non utili	8
2.5	Analisi della distribuzione delle variabili	9
2.6	Criticità osservate e costruzione di dataset alternativi	13
3	Implementazione dei modelli di Machine Learning	16
3.1	Split del dataset in <i>Training Set</i> e <i>Test Set</i>	16
3.2	Costruzione della pipeline di preprocessing e definizione dei modelli utilizzati	17
3.2.1	Custom Transformer	17
3.2.2	Preprocessor	19
3.2.3	Bilanciamento	20
3.3	Definizione dei modelli e delle griglie di iperparametri	20
3.4	Metriche di valutazione	22
3.5	Tuning degli iperparametri e strategia di validazione	23
3.6	Ciclo di addestramento	24
3.7	Valutazione delle prestazioni dei modelli	25
3.8	Combinazioni dataset-features-modelli	27
3.8.1	Utilizzo di tutte le features per addestrare i modelli	28
3.8.2	Utilizzo delle features demografiche e delle features di Esito per addestrare i modelli	28
3.8.3	Utilizzo delle features demografiche e delle features dei Punteggi Grezzi per addestrare i modelli	29

4	Risultati	31
4.1	Classificazione multiclasse a quattro categorie: <i>Assenza di patologia, Disturbo cognitivo lieve, Disturbo depressivo, Malattia di Alzheimer</i>	31
4.2	Classificazione multiclasse a tre categorie: <i>Assenza di patologia, Disturbo non-Alzheimer, Malattia di Alzheimer</i>	33
4.3	Classificazione binaria: presenza vs assenza di disturbo	34
4.4	Classificazione multiclasse a tre categorie: <i>Disturbo cognitivo lieve, Disturbo depressivo, Malattia di Alzheimer</i>	35
5	Conclusioni	37
	Bibliografia	38

Capitolo 1

Introduzione

La valutazione dei disturbi cognitivi negli adulti e negli anziani rappresenta una sfida clinica importante, in quanto le condizioni patologiche possono manifestarsi con sintomi diversi e sovrapponibili. Tra le condizioni più comuni vi sono:

- **Alzheimer**: malattia neurodegenerativa progressiva caratterizzata da un declino della memoria episodica, delle funzioni esecutive e, nelle fasi più avanzate, dell'autonomia funzionale [3].
- **Disturbi depressivi** che possono determinare deficit cognitivi secondari, spesso reversibili [6].
- Forme di **deterioramento cognitivo lieve**, che si posizionano nel mezzo tra l'invecchiamento fisiologico e la demenza, in cui si osserva un deficit cognitivo moderato che permette una certa autonomia nelle attività quotidiane [4].

La diagnosi precoce e accurata di queste patologie, in particolare dell'Alzheimer, è fondamentale per garantire interventi terapeutici tempestivi.

Per valutare le capacità cognitive, in ambito clinico, sono spesso utilizzati test neuropsicologici standardizzati. Tra essi, il **Mini Mental State Examination** (MMSE) misura in modo generale le funzioni cognitive e permette di distinguere rapidamente tra persone sane e persone con deficit cognitivi [2]; il **Clock Drawing Test**, che consiste nel disegnare un orologio indicando uno specifico orario, fornisce informazioni sulle abilità visuospatiali e sulle funzioni esecutive [8]; il **Rey Auditory Verbal Learning Test** e i **Test della figura di Rey** valutano quantitativamente rispettivamente la memoria verbale e quella visuospatial, chiedendo al paziente di ricordare e richiamare una lista di parole nel primo caso, e di copiare e poi riprodurre una figura complessa nel secondo caso [5].

I risultati di questi e di altri test possono essere utilizzati come features per allenare modelli di Machine Learning in grado di distinguere tra diverse condizioni patologiche, come la depressione e l'Alzheimer. L'uso di tali modelli può supportare i medici nella diagnosi, accelerando l'identificazione precoce della malattia e migliorando la precisione delle decisioni cliniche.

In questo elaborato viene presentato un progetto di Machine Learning basato su dati raccolti da test neurologici effettuati su pazienti. L'obiettivo principale è valutare l'efficacia di specifici modelli predittivi nel distinguere tra quattro gruppi clinici: soggetti sani, pazienti con disturbo cognitivo lieve, pazienti affetti da depressione e individui con diagnosi di malattia di Alzheimer. Verranno spiegati il dataset e la sua elaborazione, i modelli utilizzati e i principali risultati ottenuti.

Capitolo 2

Analisi e preprocessing del dataset clinico

Il dataset analizzato contiene informazioni anagrafiche e cliniche di pazienti sottoposti ad una serie di test neuropsicologici che indagano diversi aspetti delle funzioni cognitive, tra cui la memoria e l'attenzione.

In questo capitolo, sono descritti il dataset nel dettaglio e le iniziali operazioni di preprocessing (realizzate in un Notebook dedicato) effettuate su di esso, al fine di prepararlo per l'addestramento di modelli di Machine Learning.

2.1 Strumenti software utilizzati

Per realizzare il progetto è stato utilizzato Google Colab (logo in Figura [2.1](#)), strumento software che permette di scrivere ed eseguire codice Python direttamente online.



Figura 2.1: Logo Google Colab

2.2 Descrizione del dataset

Il dataset in esame è costituito da 511 righe e 41 colonne; ciascuna riga corrisponde ad un soggetto univocamente identificato da un ID. In Tabella [2.1](#) sono riportate tutte le features del dataset e la relativa descrizione.

Feature	Descrizione
ID	Codice univoco paziente

Feature	Descrizione
CAP	Codice di Avviamento Postale
Scolarità	Anni di istruzione
Età	Età al momento della somministrazione dei test
Sesso	Sesso del paziente
MMSE_PG	Punteggio grezzo associato al Mini Mental State Examination
MMSE_PC	Punteggio percentile associato al Mini Mental State Examination
MMSE_PE	Punteggio equivalente associato al Mini Mental State Examination
MMSE_ESITO	Giudizio qualitativo associato al Mini Mental State Examination
CLOCKTEST_PG	Punteggio grezzo associato al Clock Test
CLOCKTEST_PE	Punteggio equivalente associato al Clock Test
CLOCKTEST_ESITO	Giudizio qualitativo associato al Clock Test
COPIAFIGURAREY_PG	Punteggio grezzo associato all'abilità grafico-visiva nel test della figura di Rey
COPIAFIGURAREY_PC	Punteggio percentile associato all'abilità grafico-visiva nel test della figura di Rey
COPIAFIGURAREY_PE	Punteggio equivalente associato all'abilità grafico-visiva nel test della figura di Rey
COPIAFIGURAREY_ESITO	Giudizio qualitativo associato all'abilità grafico-visiva nel test della figura di Rey
PAROLEREYIMM_PG	Punteggio grezzo associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo immediato
PAROLEREYIMM_PC	Punteggio percentile associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo immediato
PAROLEREYIMM_PE	Punteggio equivalente associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo immediato
PAROLEREYIMM_ESITO	Giudizio qualitativo associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo immediato
PAROLEREYDIFF_PG	Punteggio grezzo associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo differito

Feature	Descrizione
PAROLEREYDIFF_PC	Punteggio percentile associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo differito
PAROLEREYDIFF_PE	Punteggio equivalente associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo differito
PAROLEREYDIFF_ESITO	Giudizio qualitativo associato al Rey Auditory Verbal Learning Test (RAVLT) – richiamo differito
MEMORIAFIGURAREY_PG	Punteggio grezzo associato alla memoria visuo-spaziale nel test di Rey
MEMORIAFIGURAREY_PC	Punteggio percentile associato alla memoria visuo-spaziale nel test di Rey
MEMORIAFIGURAREY_PE	Punteggio equivalente associato alla memoria visuo-spaziale nel test di Rey
MEMORIAFIGURAREY_ESITO	Giudizio qualitativo associato alla memoria visuo-spaziale nel test di Rey
FAB_PG	Punteggio grezzo associato al Frontal Assessment Battery
FAB_PC	Punteggio percentile associato al Frontal Assessment Battery
FAB_PE	Punteggio equivalente associato al Frontal Assessment Battery
FAB_ESITO	Giudizio qualitativo associato al Frontal Assessment Battery
FLUENZAVERBFON_PG	Punteggio grezzo associato al test sulla fluenza verbale
FLUENZAVERBFON_PC	Punteggio percentile associato al test sulla fluenza verbale
FLUENZAVERBFON_PE	Punteggio equivalente associato al test sulla fluenza verbale
FLUENZAVERBFON_ESITO	Giudizio qualitativo associato al test sulla fluenza verbale
TESTMATRICIATTENTIVE_PG	Punteggio grezzo associato al test sull'attenzione e ragionamento logico
TESTMATRICIATTENTIVE_PC	Punteggio percentile associato al test sull'attenzione e ragionamento logico
TESTMATRICIATTENTIVE_PE	Punteggio equivalente associato al test sull'attenzione e ragionamento logico

Feature	Descrizione
TESTMATRICIATTENTIVE_ESITO	Giudizio qualitativo associato al test sull'attenzione e ragionamento logico
Patologia	Diagnosi clinica

Tabella 2.1: Features del dataset con descrizione

Ogni test effettuato è riportato con:

- Punteggio grezzo (*_PG*): punteggio ottenuto direttamente dal test;
- Punteggio percentile (*_PC*): confronto con i punteggi della popolazione normale;
- Punteggio equivalente (*_PE*): punteggio standardizzato su scala clinica.
- Giudizio qualitativo (*_ESITO*).

Le variabili *ID*, *_ESITO*, *Sesso*, *Patologia* sono categoriche, le restanti sono tutte numeriche. In particolare, le classi delle colonne *ESITO*, a valle di operazioni fatte per uniformare, sono:

- NORMA: il punteggio indica che la prestazione rientra nei valori tipici della popolazione;
- DEFICIT: il punteggio indica che la funzione valutata è compromessa;
- BORDELIN: il punteggio si colloca al limite inferiore della norma.

Le classi della variabile target *Patologia* sono:

- Assenza di patologia;
- Disturbo cognitivo lieve;
- Disturbo depressivo;
- Malattia di Alzheimer.

2.3 Gestione dei valori mancanti

Come è possibile vedere in Figura [2.2](#), tutte le colonne, ad esclusione di *ID* e *CAP*, contengono valori mancanti e, in alcuni casi, in percentuali molto elevate.

	Missing Values	Percentage (%)
ID	0	0.00
CAP	0	0.00
Scolarita	16	3.13
Eta	137	26.81
Sesso	138	27.01
MMSE_PG	7	1.37
MMSE_PC	126	24.66
MMSE_PE	126	24.66
MMSE_ESITO	8	1.57
CLOCKTEST_PG	135	26.42
CLOCKTEST_PE	142	27.79
CLOCKTEST_ESITO	18	3.52
COPIAFIGURAREY_PG	88	17.22
COPIAFIGURAREY_PC	121	23.68
COPIAFIGURAREY_PE	63	12.33
COPIAFIGURAREY_ESITO	30	5.87
PAROLEREYIMM_PG	20	3.91
PAROLEREYIMM_PC	104	20.35
PAROLEREYIMM_PE	47	9.20
PAROLEREYIMM_ESITO	16	3.13
PAROLEREYDIFF_PG	21	4.11
PAROLEREYDIFF_PC	116	22.70
PAROLEREYDIFF_PE	50	9.78
PAROLEREYDIFF_ESITO	17	3.33
MEMORIAFIGURAREY_PG	150	29.35
MEMORIAFIGURAREY_PE	98	19.18
MEMORIAFIGURAREY_PC	148	28.96
MEMORIAFIGURAREY_ESITO	45	8.81
FAB_PG	288	56.36
FAB_PC	489	95.69
FAB_PE	344	67.32
FAB_ESITO	288	56.36
FLUENZAVERBFON_PG	72	14.09
FLUENZAVERBFON_PC	154	30.14
FLUENZAVERBFON_PE	100	19.57
FLUENZAVERBFON_ESITO	73	14.29
TESTMATRICIATTENTIVE_PG	79	15.46
TESTMATRICIATTENTIVE_PC	158	30.92
TESTMATRICIATTENTIVE_PE	109	21.33
TESTMATRICIATTENTIVE_ESITO	80	15.66
Patologia	16	3.13

Figura 2.2: Numero di valori mancanti per colonna e percentuale sul totale

Di conseguenza, per ridurre il rischio di introdurre rumore nei modelli predittivi, in una fase iniziale, si è proceduto a:

1. Eliminare le istanze prive di target (ossia senza diagnosi), poiché non utilizzabili nei modelli supervisionati.
2. Rimuovere le colonne con oltre il 50% di valori mancanti.
3. Rimuovere le righe con più del 50% di valori mancanti, per evitare di includere casi troppo incompleti.

2.4 Individuazione di possibili outliers e eliminazione di colonne non utili

Da un'attenta osservazione delle colonne relative ai punteggi percentili (*PC*) e ai punteggi equivalenti (*PE*), è emerso che questi campi contengono numerosi valori corrotti o difficili da interpretare e che presentano una percentuale di valori mancanti superiore rispetto ai corrispondenti punteggi grezzi, da cui i *PE* e i *PC* sono calcolati (Figura 2.2). Poichè la loro gestione risulterebbe troppo complessa, si è deciso di eliminare le colonne dei *PC* e *PE* e di mantenere, oltre le features demografiche, solo le colonne relative ai punteggi grezzi e ai giudizi di esito.

È stato effettuato un controllo sul range dei punteggi grezzi al fine di individuare eventuali outliers. In Figura 2.3 sono riportati i valori minimi e massimi riportati per ciascun test. Si osserva che i punteggi di tutti i test rientrano in intervalli accettabili, ad eccezione delle variabili *FLUENZAVERBFON_PG* (il cui range atteso è 0-60 [7]) e *TESTMATRICIATTENTIVE_PG* (il cui range atteso è 0-50/60 a seconda della versione del test utilizzata di cui, purtroppo, non si hanno informazioni [1]). Con l'obiettivo di avere un dataset meno rumoroso possibile, i punteggi >60 al test sulla fluenza fonemica e i punteggi >60 al test dell'attenzione sono stati eliminati e sostituiti con NaN.

```
MMSE_PG: min=10.0, max=30.0
CLOCKTEST_PG: min=0.0, max=10.0
COPIAFIGURAREY_PG: min=0.0, max=44.0
PAROLEREYIMM_PG: min=2.0, max=60.0
PAROLEREYDIFF_PG: min=0.0, max=12.0
MEMORIAFIGURAREY_PG: min=0.0, max=30.0
FLUENZAVERBFON_PG: min=2.0, max=64.0
TESTMATRICIATTENTIVE_PG: min=0.0, max=552.0
```

Figura 2.3: Range dei punteggi grezzi

Sono stanche eliminate anche le colonne *ID* e *CAP* perchè non contengono informazioni utili all'analisi e introducono bias.

In definitiva, le variabili utilizzate per addestrare i modelli sono 19 (Tabella 2.2) e le istanze 479.

Features utilizzate
Scolarità
Età
Sesso

Features utilizzate
MMSE_PG
MMSE_ESITO
CLOCKTEST_PG
CLOCKTEST_ESITO
COPIAFIGURAREY_PG
COPIAFIGURAREY_ESITO
PAROLEREYIMM_PG
PAROLEREYIMM_ESITO
PAROLEREYDIFF_PG
PAROLEREYDIFF_ESITO
MEMORIAFIGURAREY_PG
MEMORIAFIGURAREY_ESITO
FLUENZAVERBFON_PG
FLUENZAVERBFON_ESITO
TESTMATRICIATTENTIVE_PG
TESTMATRICIATTENTIVE_ESITO

Tabella 2.2: Features utilizzate per addestrare i modelli di ML

2.5 Analisi della distribuzione delle variabili

In Figura [2.4](#) è riportata la matrice di correlazione tra le variabili numeriche, che evidenzia un elevato livello di correlazione tra alcune di esse. Nonostante ciò, in un primo esperimento, tutte le variabili sono state mantenute perchè la presenza di correlazioni non implica necessariamente ridondanza informativa completa.

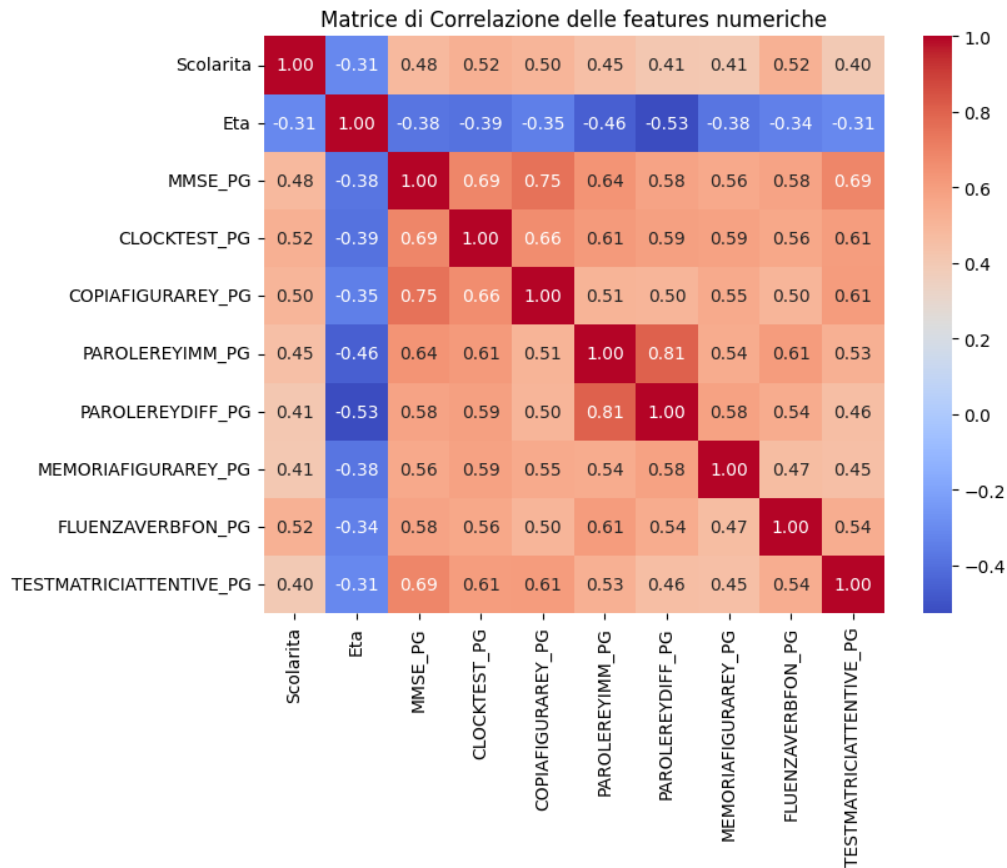


Figura 2.4: Matrice di correlazione delle variabili numeriche

Per capire come imputare correttamente in seguito i valori mancanti delle varie features, sono state analizzate le loro distribuzioni.

La distribuzione della variabile *Età* (Figura 2.5) risulta essere spostata verso destra: i missing values verranno imputati utilizzando la mediana.

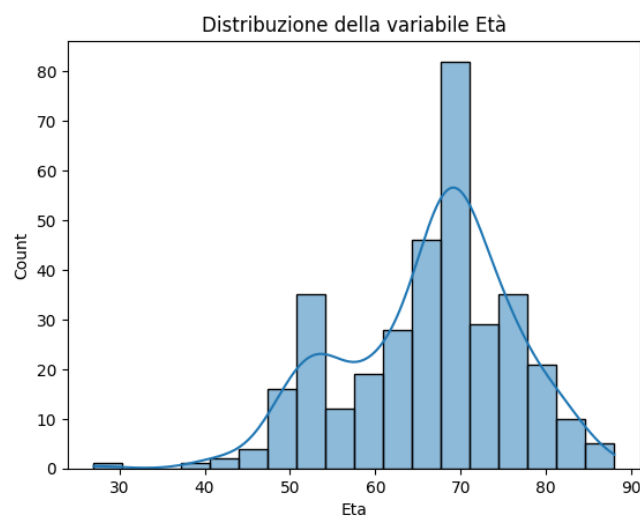


Figura 2.5: Distribuzione della variabile Età

La distribuzione della variabile *Scolarità* (Figura 2.6) non é uniforme né simmetrica: anche in questo caso i valori mancanti saranno sostituiti con la mediana.

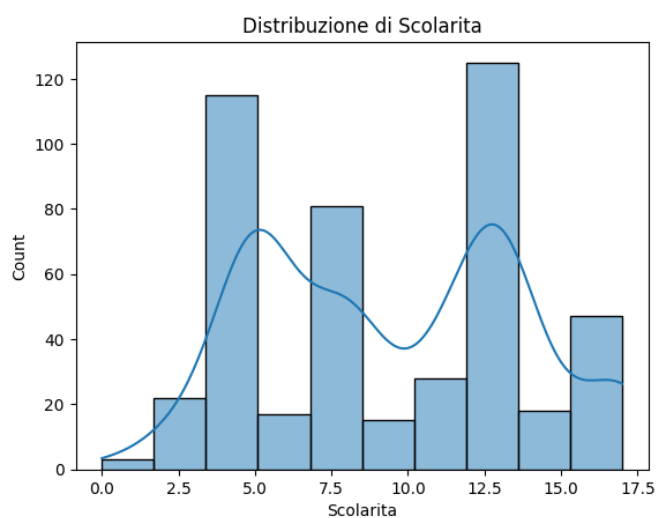


Figura 2.6: Distribuzione della variabile Scolarità

In Figura 2.7 si nota la netta predominanza di soggetti sesso femminile rispetto quelli di sesso maschile: i valori mancanti saranno qui imputati con il valore più frequente (moda).

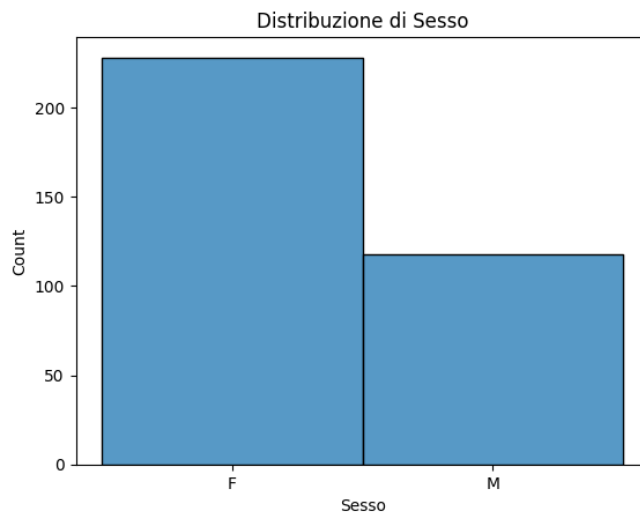


Figura 2.7: Distribuzione variabile Sesso

Data la logica relazione che sussiste tra i punteggi grezzi e le valutazioni di esito per ogni test, per imputare i valori mancanti nelle colonne *PG* sono state utilizzate le classi delle corrispettive colonne *ESITO*. I *violin plot* generati, di cui un esempio è riportato in Figura 2.8, permettono di visualizzare la distribuzione dei punteggi grezzi per ciascun gruppo di esito (la larghezza indica dove si concentrano più osservazioni) e le statistiche riassuntive (mediana, primo e terzo quartile evidenziati dalle linee tratteggiate). Le distribuzioni risultano nella maggior parte dei casi molto allungate verticalmente, suggerendo che il criterio migliore per imputare i valori mancanti di *PG* sia l'utilizzo della mediana calcolata per gruppo di esito.

Distribuzione di FLUENZAVERBFON_PG per gruppi di FLUENZAVERBFON_ESITO

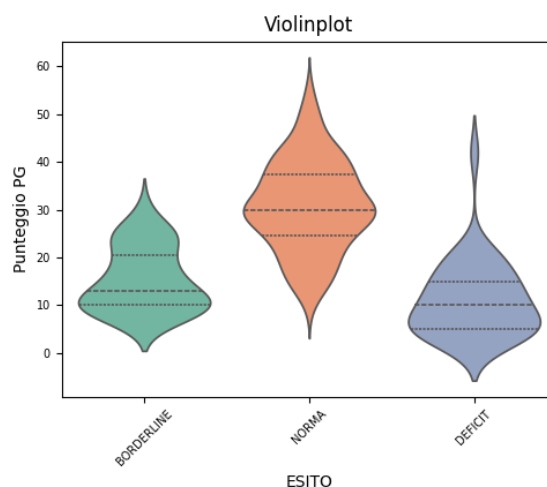


Figura 2.8: Esempio di violin plot dei punteggi grezzi rispetto i gruppi di esito

Infine, la distribuzione delle classi nella variabile target *Patologia* risulta fortemente sbilanciata, con una netta predominanza di soggetti non patologici (Figura 2.9).

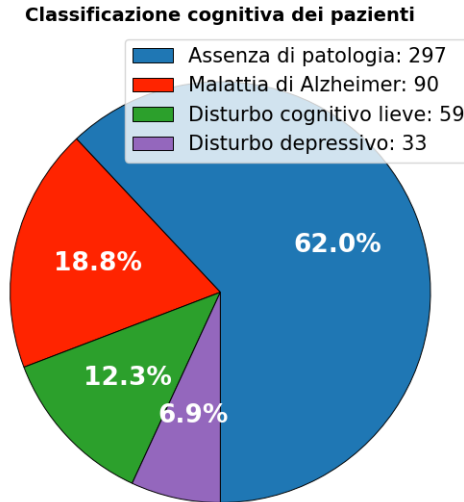


Figura 2.9: Distribuzione delle classi target

2.6 Criticità osservate e costruzione di dataset alternativi

Le principali criticità osservate riguardano l'elevato numero di valori mancanti nella maggior parte delle colonne (Figura 2.2) e il forte sbilanciamento delle classi nella variabile target (Figura 2.9) che, inevitabilmente, influisce sulla performance dei modelli che tendono a concentrarsi sulla classe più frequente penalizzando quelle più rare.

Come si vedrà meglio nel Capitolo 4, i modelli iniziali hanno ottenuto prestazioni non soddisfacenti e si è ipotizzato che una delle cause fosse la scarsa numerosità delle istanze appartenenti alle classi *Disturbo lieve* e *Disturbo depressivo*.

Per semplificare il problema di classificazione e migliorare la capacità predittiva dei modelli soprattutto nell'individuazione della classe di maggiore interesse clinico, ossia l'Alzheimer, sono stati generati altri 3 dataset alternativi:

1. Un dataset in cui la variabile target ha 3 classi (Figura 2.10):
 - **Assenza di patologia**,
 - **Disturbo non Alzheimer** (derivante dall'unione di *Disturbo lieve* e *Disturbo depressivo*),
 - **Malattia di Alzheimer**.
2. Un dataset con una versione binaria della variabile target, con l'obiettivo di discriminare tra assenza e presenza di una qualsiasi forma di disturbo cognitivo (Figura 2.11):

- **Assenza di disturbo** che include la classe *Assenza di patologia*,
 - **Presenza di disturbo** che include le istanze classificate come *Disturbo cognitivo lieve*, *Disturbo depressivo* e *Malattia di Alzheimer*.
3. Un dataset in cui sono stati eliminati i soggetti classificati come non patologici (Figura 2.12) per concentrarsi unicamente sulla classificazione tra:
- **Disturbo cognitivo lieve**,
 - **Disturbo depressivo**,
 - **Malattia di Alzheimer**.

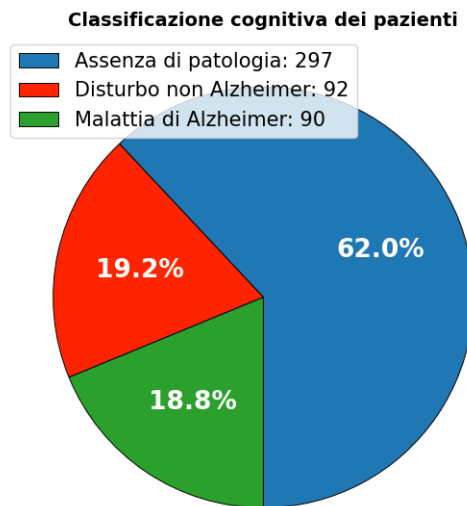


Figura 2.10: Distribuzione delle classi target nel dataset in cui vi è un'unica classe per i soggetti patologici non malati di Alzheimer

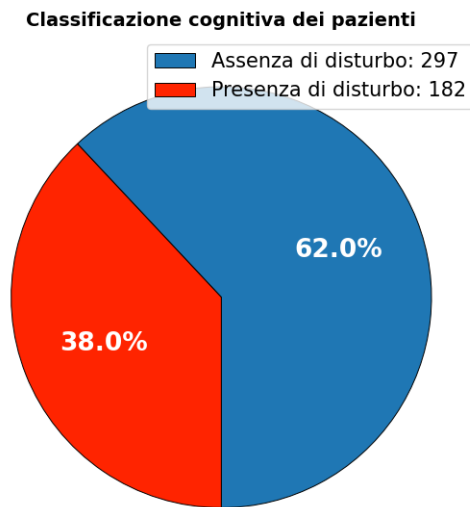


Figura 2.11: Distribuzione delle classi target nel dataset in cui si discrimina tra soggetti patologici e non patologici

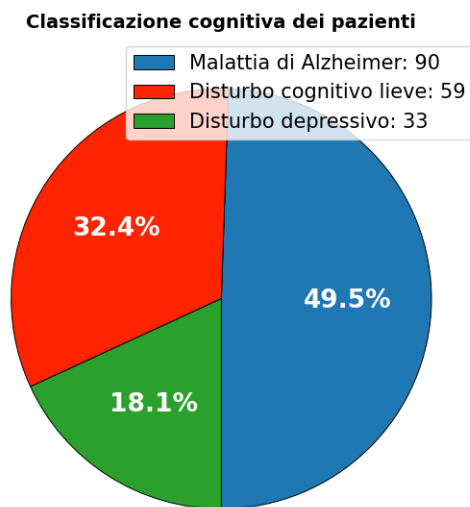


Figura 2.12: Distribuzione delle classi target nel dataset senza i soggetti patologici

L'analisi è stata dunque condotta su quattro dataset differenti: il dataset originale, con la distinzione in quattro classi patologiche, e i tre dataset alternativi appena descritti. Si noti come, in tutti i casi, i dataset siano fortemente sbilanciati.

Capitolo 3

Implementazione dei modelli di Machine Learning

In questo capitolo è mostrato l'addestramento di diversi modelli di classificazione (effettuato in un Notebook separato), con l'obiettivo di identificare quello più efficace nella predizione delle classi di disturbi neuropsichiatrici.

Si ricorda che tutte le operazioni in seguito descritte sono state effettuate su *ognuno* dei 4 dataset visti nel Paragrafo [2.6](#).

3.1 Split del dataset in *Training Set* e *Test Set*

Individuata la variabile target *Patologia*, il dataset viene suddiviso in *Training Set* (80%) e Test Set (20%) per consentire l'addestramento e la valutazione del modello (Figura [3.1](#)).

```
# Separazione delle feature (X) dal target (y)
X = df.drop('Patologia', axis=1) # Rimuove la colonna 'Patologia' per ottenere solo le feature (X)
y = df['Patologia']             # Il target è la colonna 'Patologia'

# Divisione del dataset in training set e test set
# - stratify=y in modo che le classi siano bilanciate in test set e training set
# - test_size=0.2 in modo che l'80% del dataset venga utilizzato per addestrare il modello e il 20% per testarlo
# - random_state=42 seed per rendere la divisione riproducibile
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.2, random_state=42)
```

Figura 3.1: Split del dataset in Training Set e Test Set

3.2 Costruzione della pipeline di preprocessing e definizione dei modelli utilizzati

Per garantire coerenza e replicabilità del processo di addestramento, è stata definita una pipeline (Figura 3.2) che include:

1. Imputazione dei valori mancanti,
2. Codifica delle variabili categoriche,
3. Normalizzazione delle variabili numeriche,
4. Tecniche di bilanciamento del dataset,
5. Modello di classificazione.

```
# Funzione che costruisce una pipeline completa con:
# - imputazione valori mancanti
# - preprocessing
# - bilanciamento con SMOTETomek
# - classificatore finale
def build_pipeline(classifier):
    return ImbPipeline(steps=[
        ('categorical_esito_imputer', CategoricalESITOImputer()), # Imputa *_ESITO
        ('pg_esito_imputer', PGesitoImputer()), # Imputa *_PG con mediane per ESITO
        ('iterative_pg_imputer', IterativePGImputer()), # Imputa *_PG mancanti con RF
        ('preprocessor', preprocessor), # Preprocessing variabili numeriche e categoriche
        ('sampling', SMOTETomek(random_state=42)), # Bilanciamento classi
        ('classifier', classifier) # Classificatore finale (es. DecisionTree, SVC ecc.)
    ])
```

Figura 3.2: Funzione utilizzata per definire la pipeline completa

L'utilizzo della pipeline garantisce che tutte le trasformazioni vengano eseguite nello stesso ordine e con gli stessi parametri sia durante l'addestramento che durante il test, evitando errori come lo *scaling* differente o il *data leakage* derivante dall'uso di statistiche calcolate sull'intero dataset. Tutti i passaggi vengono fittati *solo* sul training set, anche nei fold della cross-validation, garantendo che informazioni del test non influenzino il modello.

3.2.1 Custom Transformer

In un primo esperimento, per addestrare i modelli, sono state utilizzate tutte le features del dataset. Nello specifico, sono stati implementati tre *Custom Transformer* per gestire i valori mancanti delle colonne (in accordo con le osservazioni fatte nel Paragrafo 2.5):

- **CategoricalESITOImputer**: sostituisce i valori mancanti nelle colonne categoriche di *ESITO* con la moda (Figura 3.3).

```

# Imputer per colonne categoriche *_ESITO usando la moda (valore più frequente)
class CategoricalESITOImputer(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        # Seleziona tutte le colonne *_ESITO
        self.esito_cols = [c for c in X.columns if c.endswith('_ESITO')]
        # Crea un SimpleImputer per ognuna (basato su moda)
        self.imputers = {
            col: SimpleImputer(strategy='most_frequent').fit(X[[col]])
            for col in self.esito_cols
        }
        return self

    def transform(self, X):
        check_is_fitted(self, 'imputers')
        X = X.copy()
        # Applica l'imputazione a ogni colonna
        for col, imputer in self.imputers.items():
            if col in X.columns:
                X[[col]] = imputer.transform(X[[col]])
        return X

```

Figura 3.3: Classe per imputare le colonne *ESITO*

- **PGEsitoImputer**: imputa le colonne numeriche dei Punteggi Grezzi in base alla mediana calcolata per ciascuna categoria associata nella rispettiva colonna *ESITO* (Figura 3.4).

```

# Classe per imputare i valori mancanti delle colonne *_PG in base alla mediana
# (o altro metodo) calcolata per ogni categoria nella colonna corrispondente *_ESITO
class PGEsitoImputer(BaseEstimator, TransformerMixin):
    def __init__(self, method='median'):
        self.method = method # Metodo di aggregazione (es. 'median', 'mean')
        self.fill_values_ = {} # Dizionario per salvare i valori di imputazione

    def fit(self, X, y=None):
        # Per ogni colonna che finisce con '_PG'
        for col in X.columns:
            if col.endswith('_PG'):
                # Trova la colonna *_ESITO corrispondente
                esito_col = col.replace('_PG', '_ESITO')
                if esito_col in X.columns:
                    # Crea sotto-dataset senza NaN
                    sub = X[[col, esito_col]].dropna()
                    # Calcola l'aggregato (es. mediana) di 'col' per ogni categoria di 'esito_col'
                    agg = sub.groupby(esito_col)[col].agg(self.method)
                    # Salva i valori di imputazione per ogni categoria
                    self.fill_values_[col] = agg

        return self

    def transform(self, X):
        # Verifica che il fit sia stato eseguito
        check_is_fitted(self, 'fill_values_')
        X = X.copy()
        # Per ogni colonna da imputare
        for pg_col, mapping in self.fill_values_.items():
            esito_col = pg_col.replace('_PG', '_ESITO')
            # Se è NaN in *_PG ma non in *_ESITO, allora è possibile imputare
            mask = X[pg_col].isna() & X[esito_col].notna()
            # Sostituendo i NaN con il valore corretto in base a *_ESITO
            X.loc[mask, pg_col] = X.loc[mask, esito_col].map(mapping)
        return X

```

Figura 3.4: Classe per imputare i *PG* in base alla mediana nel gruppo di *ESITO* associato

- **IterativePGImputer**: utilizza `IterativeImputer` con regressori `Random Forest` per stimare eventuali ulteriori valori mancanti nelle colonne *PG*, sfruttando le correlazioni tra le variabili numeriche (Figura 3.5). Questo step è fondamentale soprattutto durante la validazione incrociata, dove può accadere che, in alcuni fold di training, non siano presenti valori di *PG* per certe categorie nella variabile *ESITO*. In questi casi, il primo imputatore (*PGEsitoImputer*) non riesce a calcolare la mediana corrispondente per quella categoria, lasciando dei valori NaN.

```
# Classe per Imputer avanzato per le colonne *_PG basato su IterativeImputer con Random Forest
class IterativePGImputer(BaseEstimator, TransformerMixin):
    def __init__(self, max_iter=50, random_state=42, n_estimators=5):
        self.max_iter = max_iter
        self.random_state = random_state
        self.n_estimators = n_estimators
        self.imputer = None # Sarà inizializzato nel fit()

    def fit(self, X, y=None):
        # Seleziona solo le colonne che terminano con '_PG'
        self.pg_cols = [c for c in X.columns if c.endswith('_PG')]
        # Crea IterativeImputer con regressore RandomForest
        self.imputer = IterativeImputer(
            estimator=RandomForestRegressor(
                n_estimators=self.n_estimators,
                random_state=self.random_state
            ),
            max_iter=self.max_iter,
            random_state=self.random_state
        )
        # Addestra l'imputer sui dati non nulli
        self.imputer.fit(X[self.pg_cols])
        return self

    def transform(self, X):
        # Verifica che l'imputer sia già addestrato
        check_is_fitted(self.imputer)
        X = X.copy()
        # Esegue l'imputazione
        imputed = self.imputer.transform(X[self.pg_cols])
        # Sostituisce solo i valori mancanti con quelli imputati
        for i, col in enumerate(self.pg_cols):
            mask = X[col].isna()
            X.loc[mask, col] = imputed[mask, i]
        return X
```

Figura 3.5: Classe per imputare i *PG* con regressore `Random Forest`

3.2.2 Preprocessor

È stato costruito un preprocessor (Figura 3.6), che applica ulteriori trasformazioni specifiche:

- Imputazione con mediana e standardizzazione con *StandardScaler* delle variabili *Età* e *Scolarità*;
- Standardizzazione dei punteggi grezzi precedentemente imputati (Paragrafo 3.2.1);
- Imputazione con moda e codifica *one-hot* per le variabili *Sesso* e *Esito*.

```

# Selezione delle features
pg_features = [col for col in X.columns if col.endswith('_PG')] # features numeriche che terminano con '_PG'
numeric_features = ['Eta', 'Scolarita'] # features numeriche 'Eta' e 'Scolarita'
esito_features = [col for col in X.columns if col.endswith('_ESITO')] # features categoriche che terminano con 'ESITO'
categorical_features = ['Sesso'] + esito_features # features categoriche 'Sesso'

# Costruzione preprocessor che applica trasformazioni diverse a gruppi di colonne
preprocessor = ColumnTransformer(transformers=[
    # Trasformazioni per le features numeriche età e scolarità
    # Utilizzo di Pipeline per definire un ordine preciso
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')), # Riempie i valori mancanti con la mediana
        ('scaler', StandardScaler()) # Standardizza: media 0, dev. std 1
    ]), numeric_features),
    # Standardizzazione dei punteggi grezzi
    ('pg_scaler', StandardScaler(), pg_features),
    # Trasformazione delle features categoriche di sesso e esito
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')), # Riempie i valori mancanti con la moda
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False)) # Codifica one-hot
    ]), categorical_features)
], remainder='passthrough') # Le colonne non specificate vengono lasciate intatte

```

Figura 3.6: Preprocessor variabili categoriche e numeriche

3.2.3 Bilanciamento

Il dataset è stato bilanciato utilizzando la funzione *SMOTETomek* (come visibile in Figura 3.2) che effettua oversampling e undersampling combinando:

- **SMOTE**: *Synthetic Minority Oversampling Technique*, genera nuovi campioni sintetici della classe minoritaria interpolando esempi già presenti;
- **Tomek Links**: rimuove i campioni ambigui della classe maggioritaria, cioè quelli molto simili nello spazio delle features ma appartenenti a classi distinte.

3.3 Definizione dei modelli e delle griglie di iperparametri

I modelli utilizzati per effettuare le predizioni sono in totale 8:

1. **WeightedKNN**: una variante di *k-Nearest Neighbors* che pondera la distanza (i "vicini più vicini" contano di più), più robusta in caso di classi sbilanciate.
2. **DecisionTree**: albero di decisione classico.
3. **BalancedRandomForest**: variante del modello *Random Forest* che bilancia automaticamente i campioni minoritari.
4. **SVM**: *Support Vector Machine*.
5. **ExtraTrees**: insieme di alberi decisionali che usa più casualità per evitare l'overfitting.

6. **LGBM**: *Light Gradient Boosting Machine*, un algoritmo gradient boosting molto efficiente.
7. **CatBoost**: algoritmo di gradient boosting che funziona bene con dati misti e gestisce bene le variabili categoriche.
8. **XGBoost**: *Extreme Gradient Boosting*, algoritmo di gradient boosting molto performante.

Questi modelli sono stati inclusi all'interno di un dizionario e a ciascuno di essi è stata associata una *griglia di iperparametri* da ottimizzare tramite la *Grid Search Cross Validation* (Figura [3.7](#)). Particolare attenzione è stata posta a settare le impostazioni dei modelli che potessero rafforzare il bilanciamento delle classi effettuato con *SMOTETomek*, in modo da ottenere risultati migliori.


```

# Dizionario dei modelli e delle relative griglie di iperparametri da ottimizzare.
# Ogni elemento contiene:
# - 'model': il classificatore (può essere None e definito nel ciclo)
# - 'param_grid': griglia di iperparametri da testare con GridSearchCV
models = {
    'weightedKNN':{
        'model': KNeighborsClassifier(weights='distance'),
        'param_grid': {
            'classifier__n_neighbors': [3, 5, 7, 9, 11, 13], # Numero di elementi vicini considerati
            'classifier__metric': ['minkowski', 'euclidean', 'manhattan'], # Metrica di distanza utilizzata
            'classifier__p': [1, 2], # utilizzato solo in caso di distanza "minkowski"
        }
    },
    'DecisionTree':{
        'model': DecisionTreeClassifier(class_weight='balanced', random_state=42),
        'param_grid': {
            'classifier__max_depth': [5, 10, 15, None], # Massima profondità dell'albero
            'classifier__min_samples_split': [5, 10, 15], # Numero minimo di campioni richiesti per dividere un nodo
            'classifier__min_samples_leaf': [2, 4, 8], # Numero minimo di campioni in una foglia (nodo finale)
            'classifier__criterion': ['gini', 'entropy'], # Funzione per valutare la qualità di una divisione
        }
    },
    'BalancedRandomForest':{
        'model': BalancedRandomForestClassifier(random_state=42, replacement=True),
        'param_grid': {
            'classifier__n_estimators': [50, 100, 200], # numero di alberi
            'classifier__max_depth': [None, 10, 20, 30], # massima profondità di ogni albero
            'classifier__min_samples_split': [2, 5, 10],
            'classifier__min_samples_leaf': [1, 2, 4],
        }
    },
    'SVM':{
        'model': SVC(class_weight='balanced', probability=True, random_state=42),
        'param_grid': {
            'classifier__C': [0.1, 1, 10], # parametro di regolarizzazione
            'classifier__kernel': ['linear', 'rbf', 'poly'], # funzione kernel utilizzata
            'classifier__gamma': ['scale', 'auto'], # controlla quanto influisce un singolo punto sulla decisione (solo per 'rbf' e 'poly')
            'classifier__degree': [3, 4], # imposta il grado del polinomio (solo per 'poly')
        }
    },
    'ExtraTrees': {
        'model': ExtraTreesClassifier(random_state=42, class_weight='balanced'),
        'param_grid': {
            'classifier__n_estimators': [50, 100, 150, 200],
            'classifier__max_depth': [None, 5, 10, 15],
            'classifier__min_samples_split': [5, 10, 15],
            'classifier__min_samples_leaf': [1, 2, 4],
        }
    },
    'LGBM':{
        'model': LGBMClassifier(class_weight='balanced', random_state=42, verbose=-1),
        'param_grid': {
            'classifier__n_estimators': [50, 100, 150, 200],
            'classifier__learning_rate': [0.01, 0.05, 0.1], # parametro che misura quanto ogni nuovo albero corregge l'errore dei precedenti
            'classifier__max_depth': [-1, 3, 5, 7, 10],
        }
    },
    'CatBoost':{
        'model': None, # sarà definito nel ciclo
        'param_grid':{
            'classifier__iterations': [300, 500], # numero di alberi
            'classifier__depth': [4, 8],
            'classifier__learning_rate': [0.01, 0.05],
        }
    },
    'XGB':{
        'model': None, # sarà definito nel ciclo
        'param_grid': {
            'classifier__n_estimators': [100, 200, 300],
            'classifier__max_depth': [3, 6, 10],
            'classifier__learning_rate': [0.01, 0.05, 0.1],
            'classifier__subsample': [0.8, 1.0], # Percentuale di dati usati per costruire ogni albero
            'classifier__colsample_bytree': [0.8, 1.0], # Percentuale di feature usate per ogni albero
        }
    }
}

```

Figura 3.7: Dizionario dei modelli utilizzati e rispettive griglie di iperparametri

3.4 Metriche di valutazione

Per valutare le prestazioni dei modelli sono state considerate (Figura [3.8](#)):

- **Balanced Accuracy**, che calcola la media del recall per ogni classe, dando lo stesso peso a tutte e evitando che il modello favorisca solo la classe maggioritaria.

$$\text{Balanced Accuracy} = \frac{1}{K} \sum_{i=1}^K \text{Recall}_i \quad \text{Recall}_i = \frac{TP_i}{TP_i + FN_i}$$

- **F1-macro**, che calcola la media semplice degli F1-score di ciascuna classe, dando la stessa importanza ad ognuna di esse.

$$F1_{\text{macro}} = \frac{1}{K} \sum_{i=1}^K F1_i \quad F1_i = \frac{2 \cdot \text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i}$$

Dove:

- K = numero di classi
- TP_i = veri positivi della classe i
- FP_i = falsi positivi della classe i
- FN_i = falsi negativi della classe i

```
# Metriche di valutazione da usare nella ricerca a griglia
scoring = {
    'balanced_accuracy': make_scorer(balanced_accuracy_score), # accuracy corretta per classi sbilanciate
    'f1_macro': make_scorer(f1_score, average='macro')      # media semplice degli F1 di ogni classe
}
```

Figura 3.8: Metriche di valutazione per la ricerca dei migliori iperparametri

3.5 Tuning degli iperparametri e strategia di validazione

Per ciascun modello, per trovare la combinazione di iperparametri che massimizza la **Balanced Accuracy**, è stata utilizzata la *Grid Search Cross Validation* (*GridSearchCV*) (Figura 3.10), che prova tutte le possibili combinazioni di iperparametri e sceglie quella con le prestazioni migliori. La valutazione è stata effettuata con una *Validazione Incrociata Stratificata* a 5 fold (*StratifiedKFold*) (Figura 3.9), che mantiene la proporzione delle classi uguale a quella dell'intero dataset: in ogni iterazione, quattro fold sono usati per l'addestramento e uno per la validazione, così da ottenere una stima più affidabile delle performance del modello.

```
# Strategia di validazione incrociata: StratifiedKFold mantiene la distribuzione delle classi in ogni fold
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) # 5 split -> a turno, 4 sono usati per l'addestramento e 1 per la validazione
```

Figura 3.9: Validazione Incrociata Stratificata a 5 fold

3.6 Ciclo di addestramento

In un ciclo di addestramento, per ciascun modello è stata costruita una pipeline che integra le fasi di preprocessing e classificazione ed è stata posta in ingresso alla ricerca a griglia con validazione incrociata (Figura 3.10). Per XGBoost è stato necessario codificare le etichette in formato numerico tramite *LabelEncoder* (Figura 3.11).

```
# Pipeline ottenuta combinando preprocessing e classificatore
pipeline = build_pipeline(model)

# Ricerca a griglia con validazione incrociata
search = GridSearchCV(
    estimator=pipeline,
    param_grid=m['param_grid'],
    scoring=scoring,
    refit='balanced_accuracy', # la metrica da ottimizzare per scegliere il miglior modello
    cv=cv,
    n_jobs=-1,
    verbose=2,
    return_train_score=True,
    error_score='raise'
)
```

Figura 3.10: Definizione della pipeline e della GridSearch

Infine, il modello ottimizzato è stato valutato sul Test Set (Figura 3.11), mai utilizzato in fase di addestramento, per stimare le performance reali dei modelli.

```
# XGBoost richiede etichette numeriche, quindi si applica LabelEncoder
if name == 'XGB':
    le = LabelEncoder()
    y_train_encoded = le.fit_transform(y_train) # da stringa a numeri interi
    y_test_encoded = le.transform(y_test)

    search.fit(X_train, y_train_encoded)

    class_labels = le.classes_
    evaluate_model(search, X_test, y_test, class_labels, le=le, is_multiclass=is_multiclass)
else:
    # Esegue la ricerca a griglia con validazione incrociata:
    # per ogni combinazione di iperparametri, la pipeline viene
    # addestrata e valutata sui fold definiti da StratifiedKFold.
    # Per ogni fold, la pipeline è fittata solo sul fold di training e la trasformazione è applicata al fold di validazione
    search.fit(X_train, y_train)
    # Recupera le classi target dal miglior modello trovato
    class_labels = search.best_estimator_.named_steps['classifier'].classes_
    evaluate_model(search, X_test, y_test, class_labels, is_multiclass=is_multiclass) # Valuta il modello ottimizzato sul test set non visto
```

Figura 3.11: Fit dei modelli sul Test Set

3.7 Valutazione delle prestazioni dei modelli

Per valutare le prestazioni dei modelli sul Test Set è stata utilizzata la funzione `evaluate_model` riportata in Figura 3.12. In particolare, la funzione utilizza il miglior modello trovato da *GridSearchCV* e ne mostra i parametri corrispondenti. Successivamente, calcola le predizioni sul Test Set e restituisce:

- i punteggi medi di cross-validation sulle metriche considerate (Balanced Accuracy e F1-macro);
- Balanced Accuracy, F1-macro e ROC AUC (che misura la capacità di un modello di distinguere tra classi diversi) sul Test Set;
- il *Classification Report*, che riassume precisione, recall e F1-score per ciascuna classe;
- la matrice di confusione, che permette di vedere dove il modello ha correttamente predetto la classe e dove ha sbagliato.

```

# Funzione per valutare le performance di un modello
# Mostra: migliori parametri, accuracy, F1, ROC AUC, classification report e confusion matrix
def evaluate_model(search, X_test, y_test, class_labels, le=None, is_multiclass=False):
    best_model = search.best_estimator_ # Prende il modello migliore trovato durante la GridSearchCV,
    # cioè quello con la miglior balanced accuracy sui dati di validazione.

    # Se si usa un LabelEncoder (es. classificazione multiclasse con etichette stringa)
    if le is not None:
        y_test_encoded = le.transform(y_test)
        y_pred = best_model.predict(X_test)
        y_pred_labels = le.inverse_transform(y_pred) # per stampa leggibile
        y_proba = best_model.predict_proba(X_test)
        y_test_eval = y_test_encoded
        y_pred_eval = y_pred
        y_pred_print = y_pred_labels
    else:
        y_pred = best_model.predict(X_test) # Predice le etichette del test set usando il miglior modello trovato dalla GridSearchCV
        y_proba = best_model.predict_proba(X_test) # Predice le probabilità per ciascuna classe sul test set
        y_test_eval = y_test # y_test_eval sarà usato per calcolare le metriche numeriche
        y_pred_eval = y_pred # y_pred_eval è la versione numerica delle predizioni
        y_pred_print = y_pred # y_pred_print è la versione da stampare nel classification report

    # Mostra le metriche di cross-validation migliori
    print("\n=== Best cross-validation scores ===")
    best_idx = search.best_index_
    for key in search.scoring:
        score = search.cv_results_[f'mean_test_{key}'][best_idx]
        print(f"{key}: {score:.4f}")

    print("\nBest parameters:", search.best_params_)

    # Calcolo delle metriche sul test set
    test_bal_acc = balanced_accuracy_score(y_test_eval, y_pred_eval)
    test_f1 = f1_score(y_test_eval, y_pred_eval, average='macro')

    # Calcolo ROC AUC (diverso tra binaria e multiclasse)
    if is_multiclass:
        test_roc_auc = roc_auc_score(y_test_eval, y_proba, multi_class='ovr', average='macro')
    else:
        test_roc_auc = roc_auc_score(y_test_eval, y_proba[:, 1]) # Usa la proba della classe positiva

    print("\n=== Test set scores ===")
    print(f"Balanced Accuracy: {test_bal_acc:.4f}")
    print(f"F1 Score (macro): {test_f1:.4f}")
    print(f"ROC AUC: {test_roc_auc:.4f}")

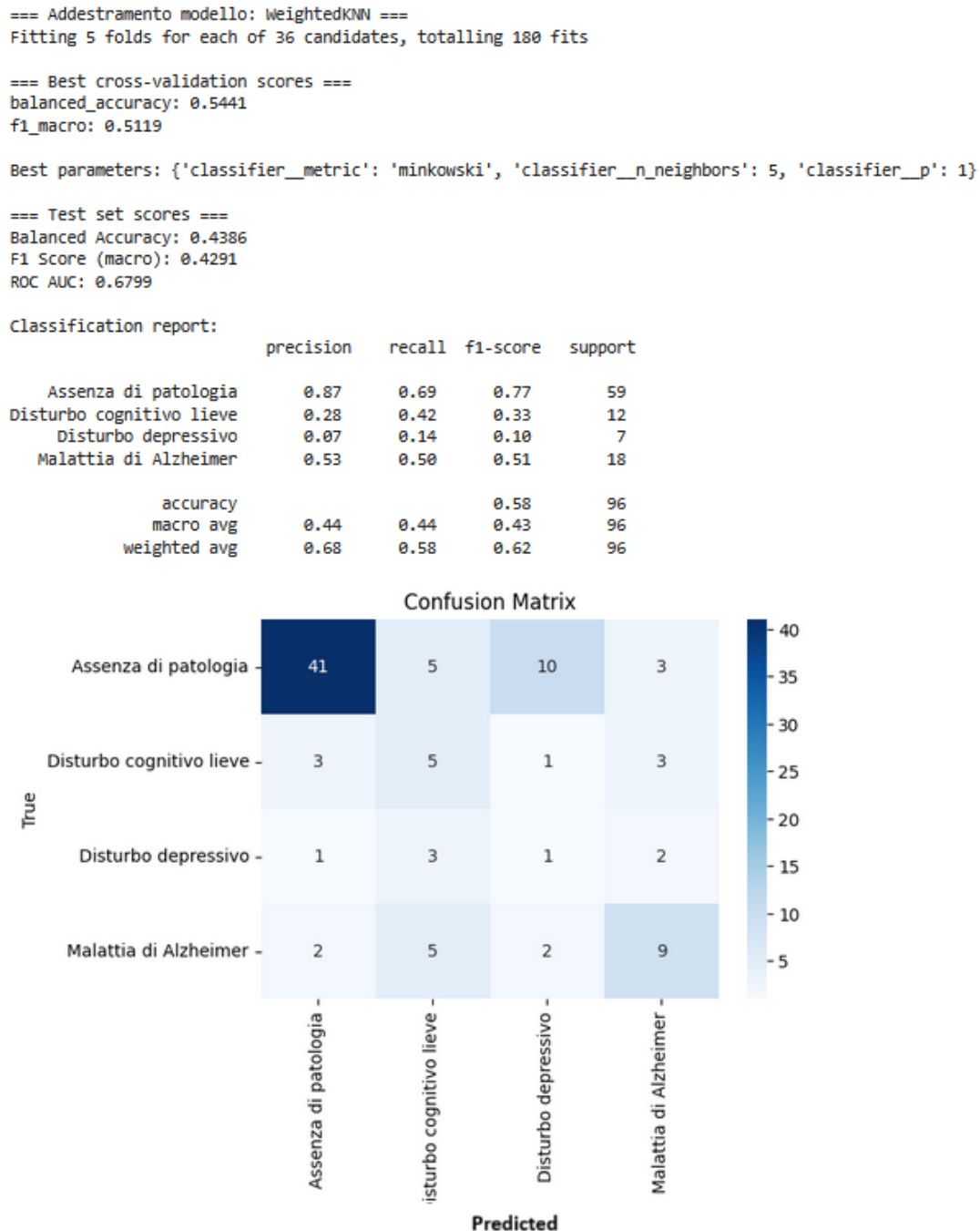
    # Stampa il classification report
    print("\nClassification report:\n", classification_report(y_test, y_pred_print))

    # Mostra la matrice di confusione con heatmap
    cm = confusion_matrix(y_test, y_pred_print, labels=class_labels)
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_labels, yticklabels=class_labels)
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.title('Confusion Matrix')
    plt.show()

```

Figura 3.12: Funzione per valutare le performance dei modelli

In Figura [3.13](#) è riportato un esempio di output della funzione `evaluate_model`.

Figura 3.13: Esempio output della funzione *Evaluate_model*

3.8 Combinazioni dataset–features–modelli

Per ciascuno dei quattro dataset generati, sono state considerate tre diverse combinazioni di feature, così da valutare come la scelta delle variabili influisce sulle prestazioni dei modelli. Ciascuna combinazione di features è stata analizzata in un notebook dedicato, in cui sono

stati raccolti i risultati dei diversi modelli. Questo approccio ha permesso di confrontare le prestazioni ottenute nelle varie configurazioni e di individuare quelle migliori.

3.8.1 Utilizzo di tutte le features per addestrare i modelli

La prima combinazione di features utilizzata per addestrare i modelli comprende tutte le features: demografiche, punteggi grezzi e valutazioni di esito. I modelli sono stati addestrati e valutati seguendo la pipeline descritta in precedenza nei Paragrafi [3.2](#) [3.7](#)

3.8.2 Utilizzo delle features demografiche e delle features di Esito per addestrare i modelli

La seconda combinazione di features utilizzata per addestrare i modelli comprende solo le features demografiche (*Età*, *Sesso*, *Scolarità*) e le features di *Esito*.

A tal proposito, sono state eliminate le colonne relative ai Punteggi Grezzi e, rispetto alla pipeline descritta nel Paragrafo [3.2](#), non sono state utilizzate le funzioni per imputare i PG mancanti (Figura [3.14](#)) e nel preprocessor non sono stati inseriti i PG (Figura [3.15](#)). Eccetto queste due modifiche, tutti gli altri passaggi hanno seguito la stessa procedura illustrata nei Paragrafi [3.2](#) [3.7](#).

```
# Funzione per la costruzione della pipeline completa
def build_pipeline(classifier):
    return ImbPipeline(steps=[
        ('categorical_esito_imputer', CategoricalESITOImputer()),
        ('preprocessor', preprocessor),
        ('sampling', SMOTETomek(random_state=42)),
        ('classifier', classifier)
    ])
```

Figura 3.14: Pipeline utilizzata nell'addestramento dei modelli con sole features demografiche e di Esito

```

# Selezione feature
numeric_features = ['Eta', 'Scolarita']
esito_features = [col for col in X.columns if col.endswith('_ESITO')]
categorical_features = ['Sesso'] + esito_features

# Costruzione preprocessor
preprocessor = ColumnTransformer(transformers=[
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), numeric_features),
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
    ]), categorical_features)
], remainder='passthrough')

```

Figura 3.15: Preprocessor della pipeline utilizzata nell'addestramento dei modelli con sole features demografiche e di Esito

3.8.3 Utilizzo delle features demografiche e delle features dei Punteggi Grezzi per addestrare i modelli

L'ultima combinazione di features con cui sono stati addestrati i modelli comprende, oltre alle features demografiche, solo i Punteggi Grezzi.

Anche in questo caso, la pipeline descritta nel Paragrafo 3.2 è stata leggermente modificata (Figura 3.16): dopo aver imputato i valori mancanti delle colonne di Esito in modo da poter imputare i Punteggi Grezzi relativi, le colonne di Esito sono state eliminate con la funzione *DropEsitoColumns* (Figura 3.17). Di conseguenza, le variabili di Esito non sono state incluse nel preprocessor (Figura 3.18). Le restanti operazioni sono analoghe a quelle viste nei Paragrafi 3.2-3.7.

```

def build_pipeline(classifier):
    return ImbPipeline(steps=[
        ('categorical_esito_imputer', CategoricalESITOImputer()), # Imputa le colonne categoriche *_ESITO
        ('pg_esito_imputer', PGEsitoImputer()), # Imputa *_PG usando *_ESITO
        ('iterative_pg_imputer', IterativePGImputer()), # Imputazione avanzata su *_PG
        ('drop_esito', DropEsitoColumns()), # Rimuove le colonne *_ESITO
        ('preprocessor', preprocessor), # Preprocessing
        ('sampling', SMOTETomek(random_state=42)), # Bilanciamento classi
        ('classifier', classifier) # Modello finale
    ])

```

Figura 3.16: Pipeline utilizzata nell'addestramento dei modelli con sole features demografiche e Punteggi Grezzi


```
# Classe per rimuovere le colonne che terminano con _ESITO
class DropEsitoColumns(BaseEstimator, TransformerMixin):
    def fit(self, X, y=None):
        # Identifica tutte le colonne con suffisso '_ESITO' da eliminare
        self.cols_to_drop_ = [c for c in X.columns if c.endswith('_ESITO')]
        return self

    def transform(self, X):
        X = X.copy()
        # Rimuove le colonne '_ESITO' dal dataset
        return X.drop(columns=self.cols_to_drop_, errors='ignore')
```

Figura 3.17: Funzione utilizzata nella pipeline per eliminare le colonne di Esito

```
# Selezione feature
pg_features = [col for col in X.columns if col.endswith('_PG')]
numeric_features = ['Eta', 'Scolarita']
categorical_features = ['Sesso']

# Costruzione preprocessor senza _ESITO
preprocessor = ColumnTransformer(transformers=[
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), numeric_features),
    ('pg_scaler', StandardScaler(), pg_features),
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
    ]), categorical_features)
], remainder='passthrough')
```

Figura 3.18: Preprocessor della pipeline utilizzata nell'addestramento dei modelli con sole features demografiche e Punteggi Grezzi

Capitolo 4

Risultati

Nel presente capitolo sono presentati i risultati dei modelli di Machine Learning applicati alle quattro versioni del dataset illustrate nel Paragrafo 2.6, considerando tre combinazioni di features per l'addestramento, come spiegato nel Paragrafo 3.8. In particolare, sono riportati solo i risultati relativi ai migliori modelli e le performance dei modelli allenati sulla combinazione di features che ha prodotto i migliori risultati globali.

4.1 Classificazione multiclasse a quattro categorie: *Assenza di patologia, Disturbo cognitivo lieve, Disturbo depressivo, Malattia di Alzheimer*

I risultati (Tabella 4.1) mostrano che la classificazione tra quattro classi è molto complessa: le performance in termini di Balanced Accuracy si aggirano intorno al 50%, con valori massimi ottenuti dal modello **Balanced Random Forest** (52%) quando addestrato su tutte le variabili (Figura 4.1). Vengono raggiunti valori accettabili di ROC AUC (>0.70), ma restano bassi F1-Score e Balanced Accuracy. In particolare, emerge una maggiore difficoltà nel riconoscere le classi di *Disturbo cognitivo lieve* e *Disturbo depressivo*, mentre la classe *Malattia di Alzheimer* risulta leggermente meglio individuata.

Modello	BALANCED ACCURACY	F1-SCORE	ROC AUC
WeightedKNN	0.4386	0.4291	0.6799
DecisionTree	0.4633	0.4686	0.7099
BalancedRandomForest	0.5217	0.5120	0.7537
SVM	0.4567	0.4449	0.7621
ExtraTrees	0.4619	0.4437	0.7304
LGBM	0.4688	0.4729	0.7703

Modello	BALANCED ACCURACY	F1-SCORE	ROC AUC
CatBoost	0.4630	0.4532	0.7985
XGB	0.4773	0.4763	0.7852

Tabella 4.1: Performance modelli allenati su variabili *Esito*, *PG*, *Età*, *Sesso* e *Scolarità*

Balanced Accuracy: 0.5217
 F1 Score (macro): 0.5120
 ROC AUC: 0.7537

Classification report:

	precision	recall	f1-score	support
Assenza di patologia	0.86	0.86	0.86	59
Disturbo cognitivo lieve	0.43	0.50	0.46	12
Disturbo depressivo	0.00	0.00	0.00	7
Malattia di Alzheimer	0.72	0.72	0.72	18
accuracy			0.73	96

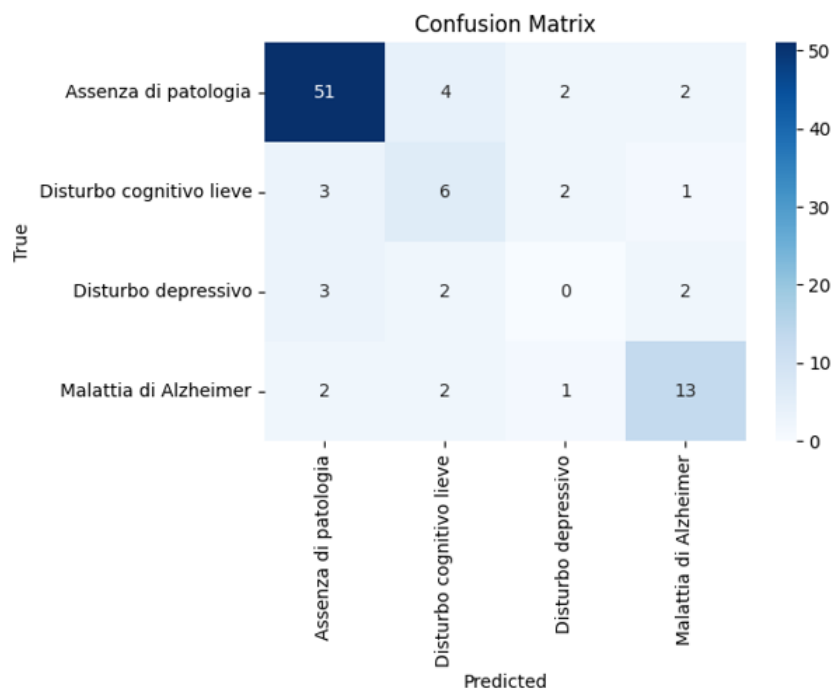


Figura 4.1: Performance del Balanced Random Forest

4.2 Classificazione multiclasse a tre categorie: *Assenza di patologia, Disturbo non-Alzheimer, Malattia di Alzheimer*

La riduzione del numero di classi porta a un netto miglioramento delle performance (Tabella 4.2): il modello che performa meglio è il **CatBoost**, addestrato sulle features demografiche e di esito, che raggiunge una Balanced Accuracy del **71%** e con ROC AUC di 0.86 (Figura 4.2). Ciò suggerisce che la distinzione tra soggetti senza patologia, con disturbo non-Alzheimer e con Alzheimer è più chiara rispetto al caso a quattro classi.

Dalle Figure 4.1 e 4.2 è possibile notare come la capacità dei modelli di riconoscere la classe Malattia di Alzheimer rimane pressoché invariata tra la classificazione a quattro classi e questa a tre classi. L'unificazione delle classi minoritarie contribuisce a ridurre la confusione complessiva (molto probabilmente dovuta anche al ridotto numero di istanze in tali classi) e determina un miglioramento delle metriche globali, senza incidere in modo sostanziale sull'accuratezza con cui viene riconosciuto l'Alzheimer.

Modello	BALANCED ACCURACY	F1-SCORE	ROC AUC
WeightedKNN	0.6093	0.5829	0.7637
DecisionTree	0.6074	0.6083	0.7378
BalancedRandomForest	0.6648	0.6706	0.8393
SVM	0.5926	0.5647	0.8045
ExtraTrees	0.6222	0.6246	0.8457
LGBM	0.6852	0.6844	0.8089
CatBoost	0.7093	0.6983	0.8602
XGB	0.6796	0.6694	0.8273

Tabella 4.2: Performance modelli allenati su variabili *Esito, Età, Sesso e Scolarità*

Balanced Accuracy: 0.7093
 F1 Score (macro): 0.6983
 ROC AUC: 0.8602

Classification report:

	precision	recall	f1-score	support
Assenza di patologia	0.91	0.85	0.88	60
Disturbo non Alzheimer	0.48	0.56	0.51	18
Malattia di Alzheimer	0.68	0.72	0.70	18
accuracy			0.77	96

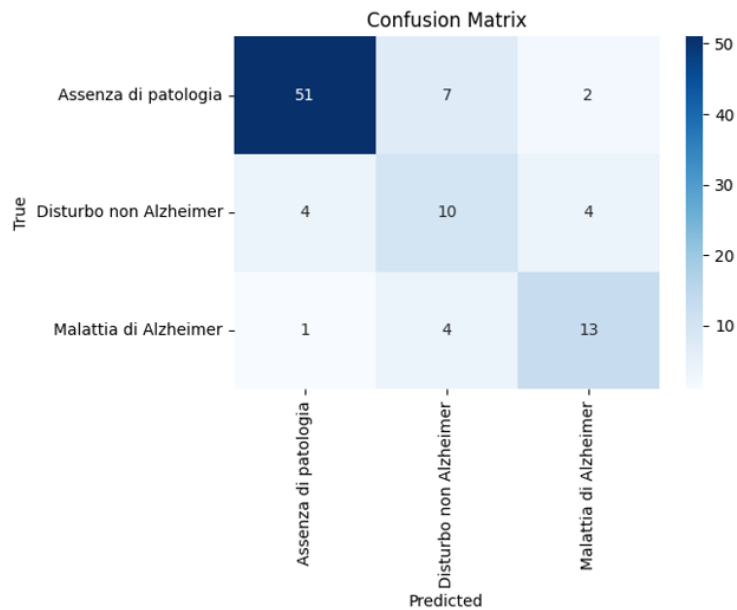


Figura 4.2: Performance del CatBoost

4.3 Classificazione binaria: presenza vs assenza di disturbo

Nel caso della classificazione binaria quasi tutti i modelli superano l'80% di Balanced Accuracy (Tabella 4.3), con il modello **XGBoost** che raggiunge valori prossimi all'89%, quando allenato su features demografiche, Esito e Punteggi Grezzi (Figura 4.3). Anche le metriche di ROC AUC (>0.90 in diversi modelli) indicano un'elevata capacità discriminativa e, in generale, tutti i classificatori riescono a distinguere bene i soggetti sani da soggetti malati. Questo conferma che la difficoltà nella classificazione aumenta all'aumentare del numero di categorie tra cui effettuare la distinzione.

Modello	BALANCED ACCURACY	F1-SCORE	ROC AUC
WeightedKNN	0.8306	0.8172	0.8880
DecisionTree	0.7833	0.7801	0.8769

Modello	BALANCED ACCURACY	F1-SCORE	ROC AUC
BalancedRandomForest	0.8861	0.8797	0.9130
SVM	0.8389	0.8273	0.9329
ExtraTrees	0.8556	0.8475	0.9102
LGBM	0.8472	0.8374	0.9005
CatBoost	0.8833	0.8704	0.9148
XGB	0.8944	0.8901	0.9083

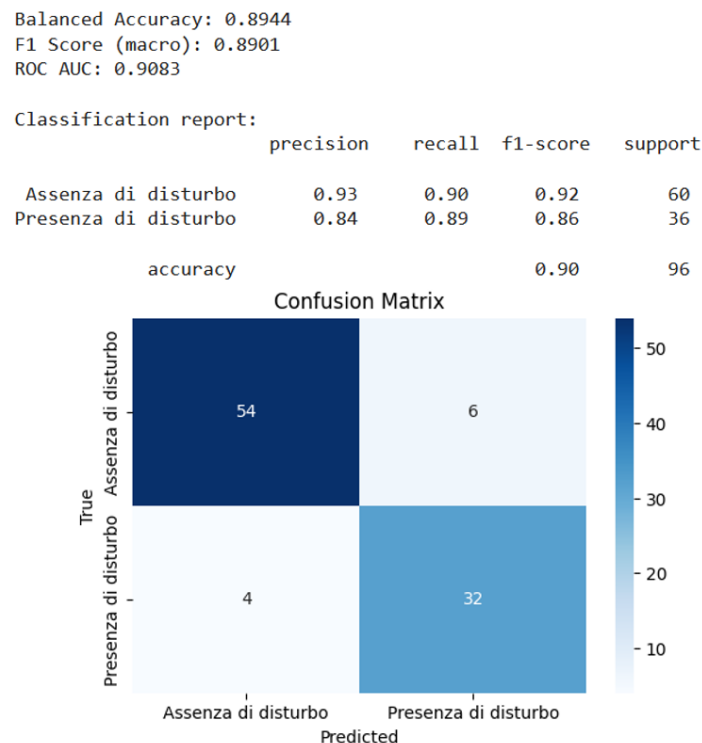
Tabella 4.3: Performance modelli allenati su variabili *Esito*, *PG*, *Età*, *Sesso* e *Scolarità*

Figura 4.3: Performance del XGBoost

4.4 Classificazione multiclasse a tre categorie: *Disturbo cognitivo lieve*, *Disturbo depressivo*, *Malattia di Alzheimer*

Nell'ultimo scenario, il focus è stato posto esclusivamente sui soggetti patologici. Il modello **ExtraTrees**, allenato sulle features demografiche e sui Punteggi Grezzi, raggiunge un valore di Balanced Accuracy del **71%** (Figura 4.4). Quasi tutti i modelli mostrano buoni risultati in termini di ROC AUC (>0.75) (Tabella 4.4). Tuttavia, continua ad esserci difficoltà nel

distinguere tra *Disturbo cognitivo lieve* e *Disturbo depressivo*. La classe Alzheimer continua ad essere quella maggiormente riconoscibile.

Modello	BALANCED ACCURACY	F1-SCORE	ROC AUC
WeightedKNN	0.6640	0.6160	0.8128
DecisionTree	0.6071	0.5935	0.6956
BalancedRandomForest	0.6627	0.6565	0.8799
SVM	0.5807	0.5439	0.7679
ExtraTrees	0.7116	0.6778	0.8458
LGBM	0.5767	0.5627	0.8336
CatBoost	0.6442	0.6286	0.8291
XGB	0.5119	0.4958	0.7559

Tabella 4.4: Performance modelli allenati su variabili *PG*, *Età*, *Sesso* e *Scolarità*

Balanced Accuracy: 0.7116
 F1 Score (macro): 0.6778
 ROC AUC: 0.8458

Classification report:

	precision	recall	f1-score	support
Disturbo cognitivo lieve	0.62	0.67	0.64	12
Disturbo depressivo	0.60	0.86	0.71	7
Malattia di Alzheimer	0.79	0.61	0.69	18
accuracy			0.68	37

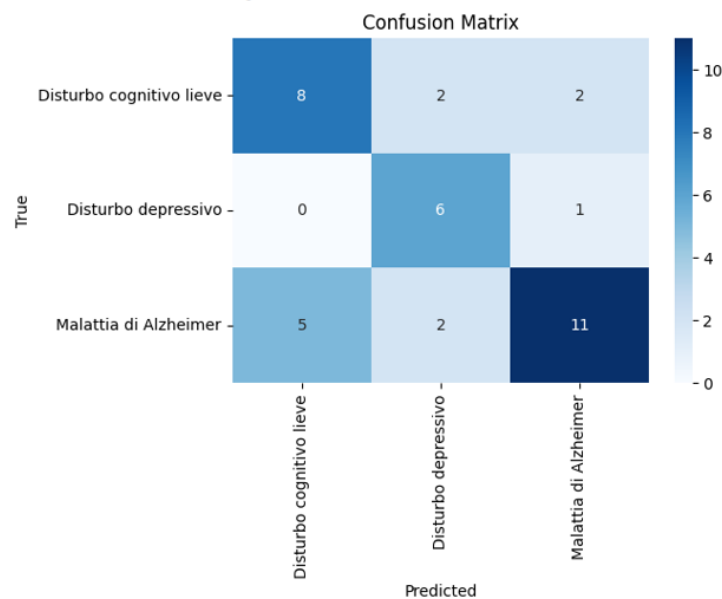


Figura 4.4: Performance del modello ExtraTrees

Capitolo 5

Conclusioni

L'analisi condotta ha permesso di valutare l'efficacia di diversi modelli di apprendimento automatico nell'ambito della classificazione di disturbi neuropsichiatrici, sulla base di dati ottenuti da test cognitivi. Attraverso un approccio sperimentale, sono stati confrontati algoritmi avanzati su differenti configurazioni di features, con l'obiettivo di identificare pattern discriminativi tra soggetti sani, pazienti con disturbo cognitivo lieve, depressione e Alzheimer.

I risultati mostrano che, nonostante le difficoltà nel distinguere i casi di disturbo cognitivo e depressivo, le cui classi risultano sottorappresentate nel dataset, la Malattia di Alzheimer viene riconosciuta con maggiore affidabilità, anche negli scenari più complessi. Questo aspetto è particolarmente rilevante dal punto di vista clinico, poiché la diagnosi precoce dell'Alzheimer rappresenta una priorità.

In particolare, i modelli che hanno performato meglio sono stati quelli più potenti e sofisticati, come XGBoost e CatBoost. La scelta di sperimentare diversi set di features si è rivelata vincente perchè ciascuno dei quattro problemi di classificazione ha raggiunto i migliori risultati con un set di caratteristiche differente.

Le ottime performance ottenute nella classificazione binaria, con una Balanced Accuracy massima pari all'89% raggiunta dal modello XGBoost, confermano il potenziale dei modelli utilizzati nell'identificare i soggetti sani da quelli patologici, ma il forte sbilanciamento del dataset potrebbe aver limitato la capacità discriminativa verso le classi meno rappresentate. In futuro, una maggiore attenzione alla costruzione di dataset più completi, un ampliamento del campione e tecniche avanzate di feature engineering potrebbero migliorare ulteriormente le prestazioni e favorire una diagnosi più accurata anche delle condizioni intermedie (Disturbo cognitivo lieve e Disturbo depressivo).

Bibliografia

- [1] Carlo Abbate, Claudio Luzzatti, and Carlo Vergani. Test delle matrici: velocità e accuratezza della ricerca visiva nel corso dell'invecchiamento matrix test: speed and accuracy of visual search in aging. *Giornale di gerontologia*, 55:11–20, 01 2007.
- [2] Miguel Gallegos, Melissa L Morgan, Mauricio Cervigni, Pablo Martino, Jessie Murray, Manuel Calandra, Anastasia Razumovskiy, Tomás Caycho-Rodríguez, and Walter Lizandro Arias Gallegos. 45 years of the mini-mental state examination (mmse): A perspective from ibero-america. *Dementia & neuropsychologia*, 16(4):384–387, 2022.
- [3] Anil Kumar, Jaskirat Sidhu, Forshing Lui, and Jack W Tsao. Alzheimer disease. In *StatPearls [internet]*. StatPearls Publishing, 2024.
- [4] Ronald C Petersen, Barbara Caracciolo, Carol Brayne, Serge Gauthier, Vesna Jelic, and Laura Fratiglioni. Mild cognitive impairment: a concept in evolution. *Journal of internal medicine*, 275(3):214–228, 2014.
- [5] Mike R Schoenberg, Kyra A Dawson, Kevin Duff, Doyle Patton, James G Scott, and Russell L Adams. Test performance and classification statistics for the rey auditory verbal learning test in selected clinical samples. *Archives of Clinical Neuropsychology*, 21(7):693–703, 2006.
- [6] Sandeep Sekhon and Raman Marwaha. Depressive cognitive disorders. *Europe PMC*, 2020.
- [7] Dolores Villalobos, Lucía Torres-Simón, Javier Pacios, Nuria Paul, and David Del Río. A systematic review of normative data for verbal fluency test in different languages. *Neuropsychology review*, 33(4):733–764, 2023.
- [8] Young Chul Youn, Jung-Min Pyun, Nayoung Ryu, Min Jae Baek, Jae-Won Jang, Young Ho Park, Suk-Won Ahn, Hae-Won Shin, Kwang-Yeol Park, and Sang Yun Kim. Use of the clock drawing test and the rey–osterrieth complex figure test-copy with convolutional neural networks to predict cognitive impairment. *Alzheimer's research & therapy*, 13(1):85, 2021.